人工智慧無線通訊
Artificial Intelligence Wireless
HW-1

Exercise 1.1

With the development of machine learning technology, its application in the field of wireless communication has gradually attracted attention. What potential applications do you see for machine learning technology in wireless communications? What challenges might machine learning face when applied to communication systems?

Answer:

Potential Applications of Machine Learning in Wireless Communications

- **Channel Estimation and Signal Detection**
  ML models can learn complex channel characteristics, improving channel estimation and symbol detection, especially in non-linear or high-mobility environments.

- **Resource Allocation and Power Control**
  Reinforcement learning can optimize spectrum allocation, beamforming, user scheduling, and power control in dynamic network conditions.

- **Network Optimization and Self-Organizing Networks (SON)**
  ML enables automated network configuration, fault detection, traffic prediction, and load balancing in 5G/6G systems.

- **Beamforming and Massive MIMO Optimization**
  Learning-based approaches can reduce computational complexity while maintaining near-optimal beam selection performance.

Challenges of Applying Machine Learning in Communication Systems

- **Data Availability and Labeling**
  High-quality labeled communication data is often difficult and expensive to obtain.
- **Generalization Across Environments**
  Wireless environments vary significantly (urban, indoor, high mobility), making trained models prone to domain shift.
- **Real-Time Constraints**
  Communication systems require ultra-low latency. Large ML models may not meet strict timing requirements.
- **Computational and Energy Constraints**
  Edge devices and base stations have limited computational resources and power budgets.
- **Interpretability and Reliability**
  Communication systems demand predictable and stable performance. Black-box models may be difficult to certify.

Exercise 1.2

Section 1.2 introduces several commonly used activation functions. Answer these questions.

(a) For hidden cells, why is the tanh activation function generally more efficient than the sigmoid activation function?

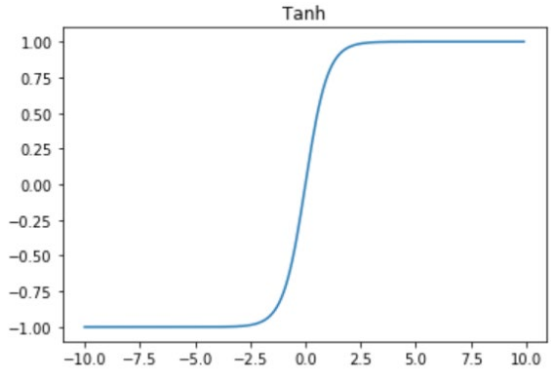
Answer:

The tanh activation function is generally more efficient than the sigmoid function for hidden layers because:

- **Zero-centered output**:
  1. Sigmoid outputs values in (0, 1), which are always positive.
  2. Tanh outputs values in (−1, 1), which are zero-centered.

     Zero-centered activations help gradients flow more symmetrically during backpropagation, leading to faster convergence.
- **Stronger gradients near zero**:

  Tanh has a steeper slope around zero compared to sigmoid, which often results in larger gradients and more efficient learning.

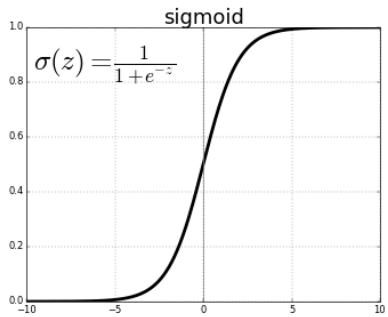


(b) Suppose you are building a binary classifier that identifies cucumbers (labeled as y = 1) and watermelons (labeled as y = 0). Which activation function would be most appropriate for the output layer, and why?

Answer:

For a binary classifier, the most appropriate activation function for the output layer is the **sigmoid** function.

- Sigmoid maps outputs to the range (0, 1).
- This can be interpreted as a probability P(y=1 | x).
- It works naturally with binary cross-entropy loss.



(c) Compute the derivative of the sigmoid function at x = 0, and prove that the derivative of the sigmoid function can be expressed in terms of the sigmoid function itself, i.e., f'(x) = f(x)(1 − f(x)).

Answer:

derivative chain rule

$$f(x) = \frac{1}{1+e^{-x}} \Rightarrow f'(x) = \frac{d}{dx}\left(\frac{1}{1+e^{-x}}\right) \Rightarrow f'(x) = \frac{e^{-x}}{(1+e^{-x})^2}$$

multiplying

$$1 - f(x) = \frac{e^{-x}}{1+e^{-x}} \Rightarrow f(x)(1-f(x)) = \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} = \frac{e^{-x}}{(1+e^{-x})^2}$$

Therefore:

$$f'(x) = f(x)(1 - f(x))$$

So, f'(0) = f(0)(1 − f(0)). = 0.5(1-0.5) = 0.25

Exercise 1.3

Answer the following questions regarding the use of CNNs in image processing.

(a) Suppose your input is a 300 × 300 color (RGB) image, and you are using a plain MLP. If the first hidden layer has 100 neurons, each fully connected to the input layer, how many parameters does this hidden layer have (including bias parameters)?

Answer:

300x300x3 = 2,700,00 (Weights per neuron)

270,000 + 1 (Bias per neuron) = 270,001

Total parameters: 100 × 270,001 = 27,000,100

Final Answer: 27,000,100 parameters

(b) Suppose your input is a 300 × 300 color (RGB) image, and you use a convolutional layer and 100 filters, each of which is 5 × 5 in size. How many parameters (including bias parameters) does this hidden layer have?

Answer:

Each filter size: 5 × 5 × 3 = 75 weights

Each filter also has 1 bias → Total per filter: 75 + 1 = 76

There are 100 filters → Total parameters: 100 × 76 = 7,600

Final Answer: 7,600 parameters

(c) Suppose you have an input of 63 × 63 × 16 and convolve it with 32 filters of size 7 × 7, using a stride of two and no padding. What is the output?

Answer:

Input size: 63 × 63 × 16

Filter size: 7 × 7

Stride = 2

Padding = 0

$$\text{Output size} = \left\lfloor \frac{N-F}{S} \right\rfloor + 1$$

So, $\frac{63-7}{2} + 1 = \frac{56}{2} + 1 = 28 + 1 = 29$

Number of filters = 32

Final Answer : 29 × 29 × 32

Exercise 1.4

Answer the following questions about CNNs.

(a) Given the input data X and the filter W of one convolutional layer, write the outputs y0 when the stride is set as S=1 and S=2.

$$X = \begin{bmatrix} 1.23 & 0.94 & 0.83 & 0.2 \\ 0.34 & 0.44 & 0.37 & 0.3 \\ 0.34 & 1.14 & 2.34 & 1.24 \\ 0.49 & 0.18 & 2.04 & 1.42 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 0.5 \\ 0.3 & 1 \end{bmatrix}$$

Answer:

When S=1, output is

```
== (a) S=1 ==
[[2.242 1.857 1.341]
 [1.802 3.307 2.462]
 [1.237 4.404 4.992]]
```

When S=2, output is

```
== (a) S=2 ==
[[2.242 1.341]
 [1.237 4.992]]
```

(b) To prevent the change of the data size, we use zero-padding and set P=1. Recalculate the outputs y0 in (a). With the stride is set as S=1, write the outputs when the y0 is processed by max-pooling and average-pooling.

Answer:

Recalculate:

```
== (b) Padding=1, S=1 ==
[[1.23  1.309 1.112 0.449 0.06 ]
 [0.955 2.242 1.857 1.341 0.29 ]
 [0.51  1.802 3.307 2.462 0.672]
 [0.66  1.237 4.404 4.992 1.666]
 [0.245 0.58  1.2   2.75  1.42 ]]
```

Max Pooling (2×2, stride=1):

```
Max Pooling:
[[2.242 2.242 1.857 1.341]
 [2.242 3.307 3.307 2.462]
 [1.802 4.404 4.992 4.992]
 [1.237 4.404 4.992 4.992]]
```

Average Pooling (2×2, stride=1):

```
Average Pooling:
[[1.434   1.63    1.18975 0.535  ]
 [1.37725 2.302   2.24175 1.19125]
 [1.05225 2.6875  3.79125 2.448  ]
 [0.6805  1.85525 3.3365  2.707  ]]
```

(c) The size of input image is 3×3×3, which consists of three 3×3matrices. And there are two filters W1,W2 in this convolutional layer. Write the outputs when the stride is set as S=1 and the zero-padding is set as P=1

$$\mathbf{R} = \begin{bmatrix} 123 & 94 & 83 \\ 34 & 44 & 37 \\ 34 & 114 & 234 \end{bmatrix}, \mathbf{G} = \begin{bmatrix} 123 & 94 & 20 \\ 11 & 30 & 22 \\ 12 & 40 & 23 \end{bmatrix}, \mathbf{B} = \begin{bmatrix} 123 & 94 & 83 \\ 34 & 44 & 37 \\ 34 & 114 & 234 \end{bmatrix},$$

$$\mathbf{W}_1 = \begin{bmatrix} 1 & 0.5 \\ 0.3 & 1 \end{bmatrix}, \mathbf{W}_2 = \begin{bmatrix} 0.5 & 1 \\ 1 & 0.3 \end{bmatrix}.$$

Answer:

```
=== (c) Multi-channel, P=1, S=1 ===
Output shape: (2, 4, 4)
[[[369.   392.7 270.6  55.8]
  [263.5 651.7 506.4 214.8]
  [119.5 430.   737.4 243.3]
  [ 40.   214.   513.5 491. ]]

 [[110.7 453.6 337.8 186. ]
  [392.7 580.9 473.8 189. ]
  [103.   317.9 570.3 539. ]
  [ 80.   308.   625.   245.5]]]
```